39. Heisenbug'lar ve Race Condition: Asenkron Hataları Yakalamak

İncelemeye Çalıştığınızda Ortadan Kaybolan Hayalet Hatalarla Mücadele
Bölüm 7 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Koda `console.log` eklediğinizde veya breakpoint koyduğunuzda sihirli bir şekilde düzelen, ama canlıda patlayan 'Heisenbug' nedir ve nasıl çözülür?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Heisenbug, gözlemlendiğinde (log veya debugger eklendiğinde gecikme değiştiği için) davranışını değiştiren veya kaybolan hatalardır. - 📌 Asenkron sistemlerde en büyük tehlike Race Condition'dır (Yarış Durumu); verinin hangi sırayla geleceği garanti edilemez. - 📌 `Promise.all` yerine sıralı bağımlılıkları `for..of` ile çözmek veya dağıtık kilitler (Redis Redlock / DB Locks) kullanmak yarış durumunu önler.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Kuantum fiziğindeki Heisenberg Belirsizlik İlkesini duymuşsunuzdur: Bir elektronun hızını ölçmeye çalıştığınız an, ölçüm aletinin ışığı elektronun yerini değiştirir!

Yazılımda da **Heisenbug** tam olarak budur: Canlı ortamda iki asenkron işlem yarışıyor. Aralarında 2 milisaniyelik bir fark var ve sistem çöküyor. Geliştirici hatayı yakalamak için araya `console.log` veya breakpoint koyuyor. Fakat o `console.log` diske yazarken sistemi 5 milisaniye yavaşlatıyor! Zamanlama değiştiği için hata birdenbire yok oluyor! Geliştirici çıldırır: *"Az önce hata veriyordu, şimdi çalışıyor!"*

Bu hayalet hatalarla baş etmenin tek yolu, şansa bırakılmış zamanlamalara değil, deterministik kilitlere ve sıralama kurallarına güvenmektir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Race Condition (Yarış Durumu) Anatomisi:

```
BAŞLANGIÇ: Kullanıcı Bakiyesi: 100 TL
İki İstek Aynı Anda Gelir (Çift Tıklama):

İSTEK 1: (50 TL Çek)                İSTEK 2: (80 TL Çek)
    |                                    |
    ├─ Bakiyeyi Oku: 100 TL              ├─ Bakiyeyi Oku: 100 TL
    ├─ Kontrol: 100 >= 50? (EVET)        ├─ Kontrol: 100 >= 80? (EVET)
    ├─ Yeni Bakiye: 50 TL                ├─ Yeni Bakiye: 20 TL
    └─ DB'ye Yaz: 50 TL                  └─ DB'ye Yaz: 20 TL
_____________________________________________________________
SONUÇ FELAKETİ:
Kullanıcı 130 TL para çekti, ama hesapta 20 TL kaldı! Banka soyuldu!
ÇÖZÜM: Atomic SQL Update: UPDATE accounts SET balance = balance - 50 WHERE balance
>= 50;
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Eşzamanlılık (Concurrency) ve Asenkron Sistemler Mimarı
Bir e-ticaret sitesinde bilet satışı yapıyoruz. Son 1 bilet kaldı ve iki kullanıcı
aynı anda 'Satın Al' butonuna bastı.

## GÖREV:
1. Bu durumdaki Race Condition tehlikesini ve 'Double-Spending' riskini açıkla.
2. PostgreSQL kullanarak 'Optimistic Locking' veya 'Atomic Update' ile bu sorunu
kökten çözen SQL / Prisma kodunu yaz.
3. Node.js tarafında `Promise.all` kullanılırken düşülen asenkron sıralama
tuzaklarını listele.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Felaket: Önce oku, sonra kontrol et, sonra yaz (Race Condition Açığı!)
export async function buyTicketNaive(ticketId: string, userId: string) {
  const ticket = await db.ticket.findUnique({ where: { id: ticketId } });

  // İki istek aynı anda burayı geçerse ikisi de bileti aldığını sanır!
  if (ticket.isSold) throw new Error("Bilet tükendi.");

  await db.ticket.update({
    where: { id: ticketId },
    data: { isSold: true, ownerId: userId }
  });
  return { success: true };
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Kurşun Geçirmez: Atomik SQL Güncellemesi (Tek İşlemde Oku ve Kilitle)
export async function buyTicketAtomic(ticketId: string, userId: string) {
  // Veritabanı seviyesinde tek satırlık koşullu güncelleme (Row-level atomic write)
  const updatedRows = await db.$executeRaw`
    UPDATE tickets
    SET is_sold = TRUE, owner_id = ${userId}, version = version + 1
    WHERE id = ${ticketId} AND is_sold = FALSE
  `;

  // Eğer etkilenen satır sayısı 0 ise, yarışı diğer kullanıcı kazanmıştır!
  if (updatedRows === 0) {
    throw new ConflictError("Bilet sizden önce başka bir kullanıcı tarafından satın
alındı.");
  }

  return { success: true, message: "Bilet başarıyla adınıza rezerve edildi." };
}
```

💡 **Kod Analizi:** `UPDATE ... WHERE is_sold = FALSE` sorgusu veritabanı motorunun dahili satır kilidini (Row-level lock) kullanır. İki istek aynı milisaniyede gelse bile veritabanı istekleri kuyruğa sokar; ilki günceller, ikincisi 0 satır etkilediği için `ConflictError` fırlatır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Asenkron döngülerde `forEach` kullanmak (forEach asenkron await'leri beklemez!).
  - **Çözüm:** Sıralı asenkron işlemlerde daima `for (const item of items)` döngüsü kullanın.
- **Tuzak:** Para veya stok gibi kritik verileri hafızada (in-memory) tutup veritabanına sonradan yazmak.
  - **Çözüm:** Kritik kaynakları daima veritabanının atomik kilitleri veya Redis mutex ile koruyun.
- **Tuzak:** Heisenbug çıktığında 'şimdi geçti bir şey olmaz' diyerek sorunu görmezden gelmek.

- **Çözüm:** Zamanlama hataları canlıda bin kat daha sık çıkar; sorunu izole edip kilit mekanizması ekleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 39: Heisenbug ve Race Condition - Asenkron Hatalarla Savaş*

- 🖥️ **Slayt 1:** Heisenbug nedir? İncelendikçe kaybolan hayalet hatalar.
- 🖥️ **Slayt 2:** Race Condition (Yarış Durumu) ve Çift Harcama (Double-Spending) tehlikesi.
- 🖥️ **Slayt 3:** JavaScript Event Loop ve asenkron döngü tuzakları (forEach vs for..of).
- 🖥️ **Slayt 4:** Veritabanı seviyesinde Atomik Kilitler (Atomic Updates) ile mutlak koruma.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Node.js'te `Promise.all` ile aynı bakiye fonksiyonuna 10 eşzamanlı istek atan bir script çalıştırın. Naive kodda bakiyenin eksiye düştüğünü gösterin. Ardından atomik SQL kodunu çalıştırıp sadece ilk isteğin geçtiğini, kalan 9'unun engellendiğini kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde bakiye düşürme, stok güncelleme veya kupon kullanma gibi kritik bir fonksiyonu inceleyin. `find` ve `update` iki ayrı adımda yapılıyorsa bunu tek bir atomik sorguya dönüştürün.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: JavaScript kodunda `[1, 2, 3].forEach(async (id) => { await doTask(id); })` yazıldığında ne olur? - A) Görevler sırayla çalışır - B) `forEach` asenkron Promise dönüşünü beklemez; tüm görevler aynı anda yarışarak asenkron kaos yaratır - C) Kod hiç çalışmaz - D) Bilgisayar kapanır > **Doğru Yanıt:** B > **Açıklama:** `Array.prototype.forEach` geriye dönen Promise'leri beklemez (await etmez). Sıralı yürütme için `for..of` döngüsü kullanılmalıdır.

Soru 2: Veritabanında eşzamanlı iki isteğin aynı anda stok düşürmesini engellemenin en zarif ve performanslı yolu nedir? - A) Sunucuyu 1 saniye bekletmek - B) `UPDATE products SET stock = stock - 1 WHERE id = 1 AND stock > 0` gibi tek satırlık atomik koşullu güncelleme kullanmak - C) Kullanıcılara aynı anda tıklamamalarını rica etmek - D) Veritabanını silmek > **Doğru Yanıt:** B > **Açıklama:** Koşullu atomik SQL güncellemesi, veritabanı motorunun dahili satır kilidini kullanarak yarış durumunu (race condition) sıfır ek maliyetle çözer.